

Event Handling

Overly Complicated Machine Example



Overly Complicated Machine Activity

1. Get in teams of 3 to 4 people.
2. One volunteer from each team gets 1 sheet of paper for scratch/brainstorming
3. As a team, brainstorm an overly complicated machine with 5 events that accomplishes some final goal.
4. Your team's machine must be drawn on the paper with the events labeled. It can be ugly but understandable enough to be able to explain it to others. **Write each team member's name on the back of the page.**

Overly Complicated Machine Activity

1. Now that everyone has an over complicated machine design, group up with one other team (so your group will have 6 to 8 people).
2. (2-5 minutes) Take turns where each team describes their overly complicated machine to the other team.

Reflections on Unplugged Activity

What was the most difficult part of designing an overly complicated machine with 5 events?

Reflections on Unplugged Activity

What if I asked you to change your machine so that one of the devices would have to be triggered by two events instead of just one?

Would this be possible?

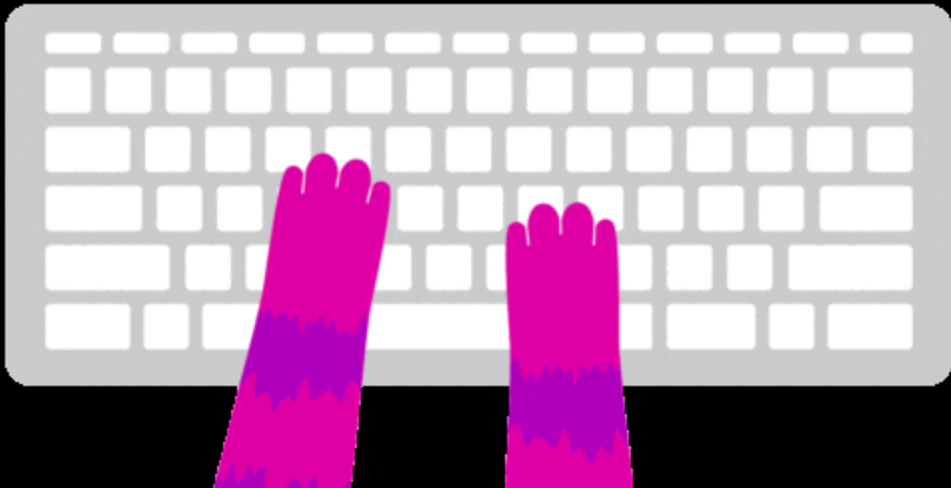
Reflections on Unplugged Activity

This activity mimics how events happen in coding.

An Event in Programming

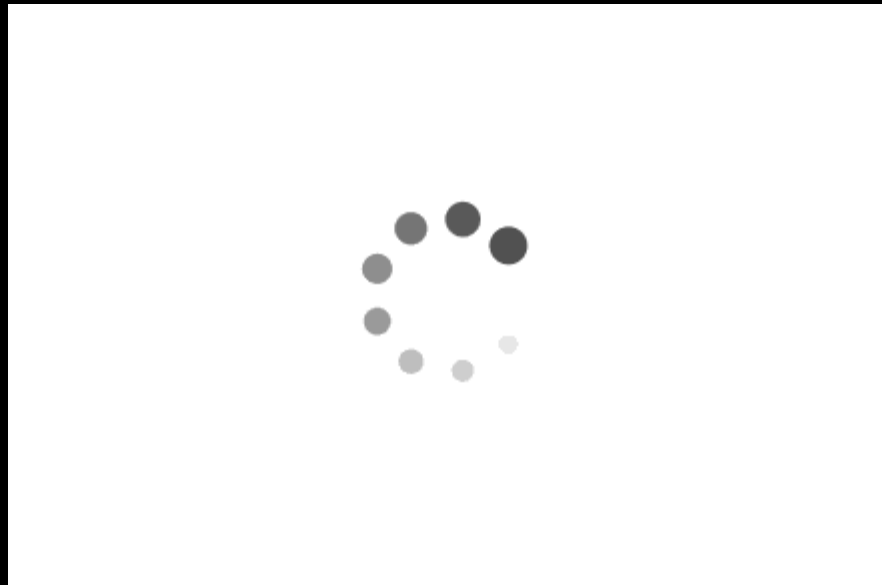
An **event** is some action that indicates a user interacting with a program. Examples of events on a GUI:

- **mouse click**
- **pressing a key on the keyboard**



An Event in Programming

A different example from HTML web pages, a webpage finishing loading is also considered an **event**.



Response to an Event

When an **event** happens, something should happen in **response** to it.

Code must be created to ensure that every event is understood, and the appropriate action fired in response.

Response to an Event

Using HTML as an example, programmers use JavaScript code to react to events such as user's clicking on website buttons or links or when the webpage finishes loading.

Code that is written that reacts to events are called **event handlers**.

Event handler code will verify user input, user actions, & browser actions so the appropriate response occurs every time a page loads, when a page is closed, when a user clicks a button, when a user inputs data into a form, and so on.

Examples of Events & Event Handlers

EVENTS

A new tab is opened in a browser

Submit button is clicked on a form

Mouse click on a screen

Total button is clicked

EVENT HANDLERS

Notification pops up on the webpage.

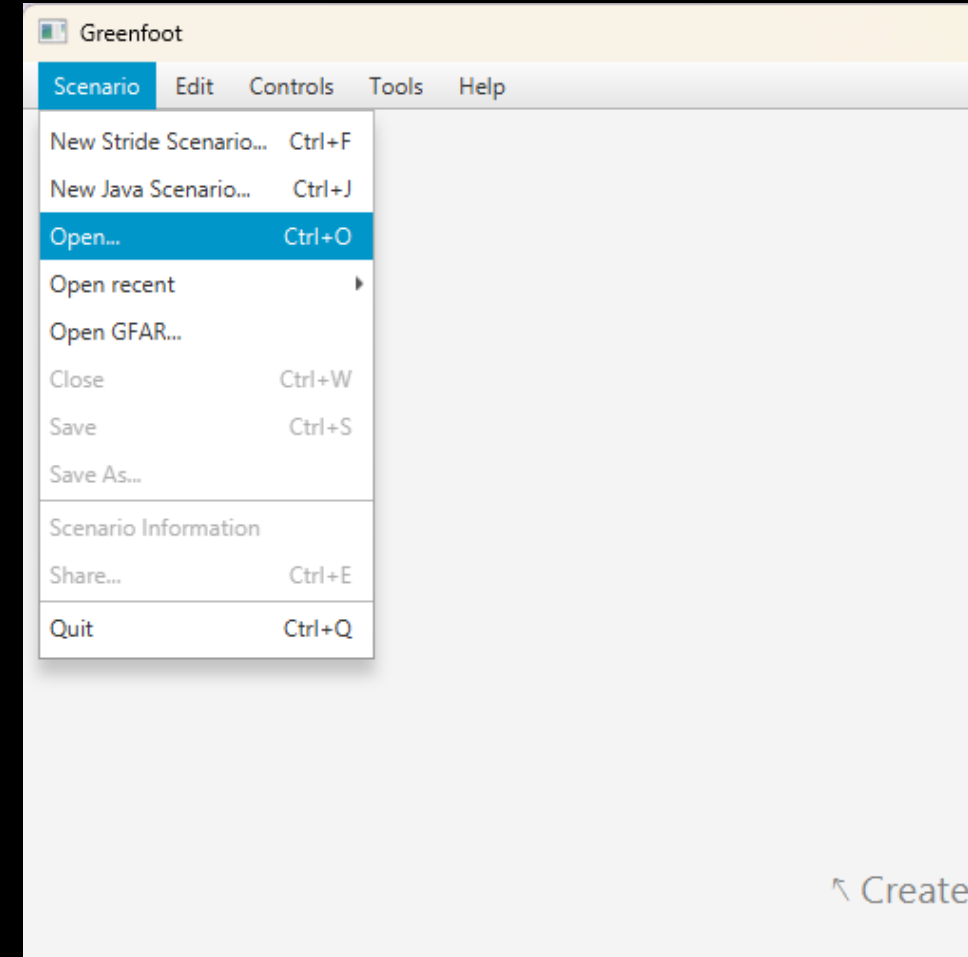
Form is submitted

Screen background color is changed

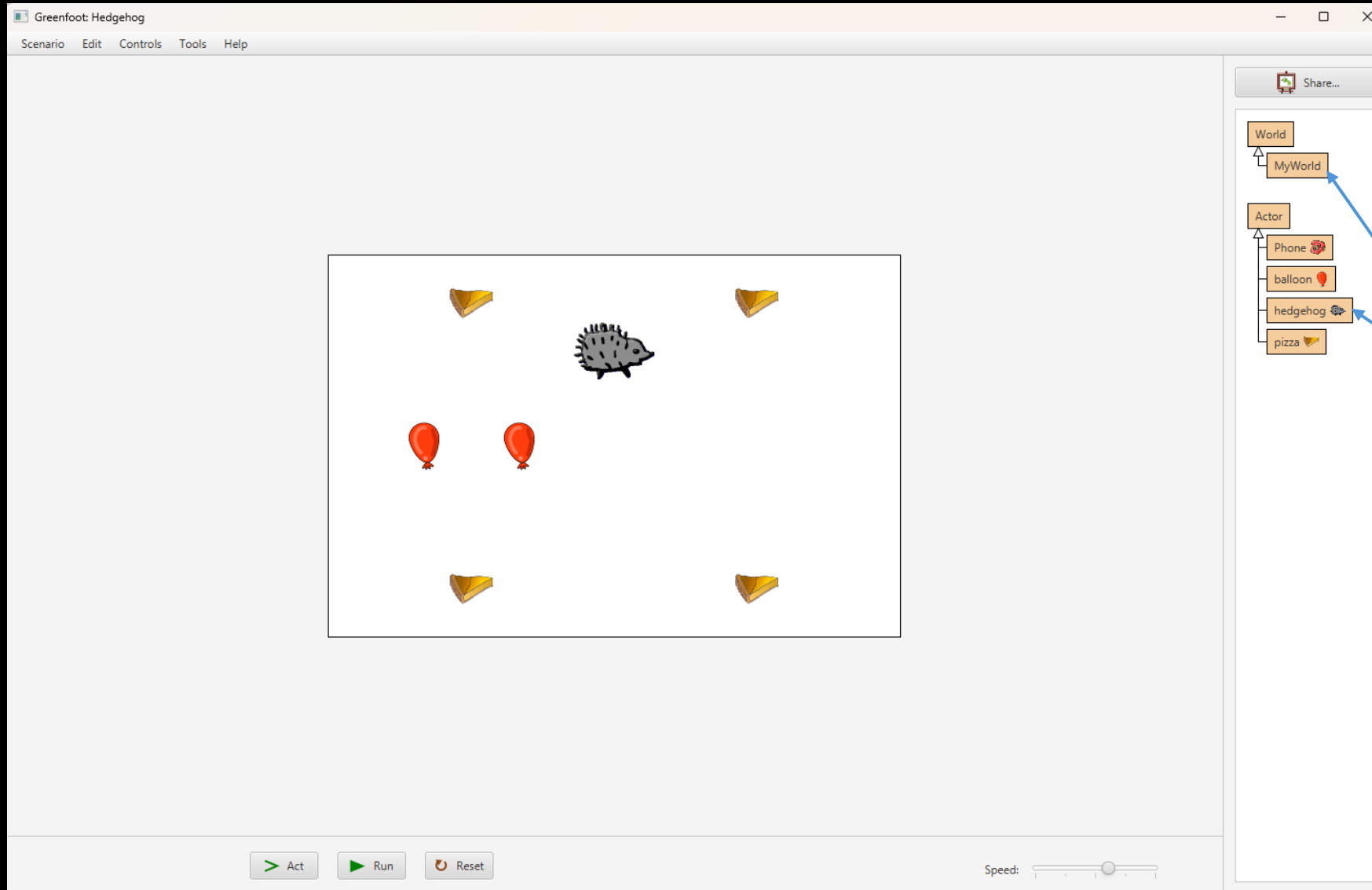
Math calculation executes and displays result on screen

Greenfoot Demo & Activity

- Step 1: install Greenfoot:
 - <https://www.greenfoot.org/download>
- Download and unzip/extract the attached zip folder
 - Hedgehog.zip
- Open the extracted Hedgehog folder in Greenfoot



Greenfoot Demo & Activity



Double click on each
element to view its
code

Hedgehog World

- MyWorld creates the basic scene
 - `super()` sets the width, height, and scale of the scene
 - `addObject()` puts objects into the scene

Hedgehog actors

balloon

`act()` method gets called every *frame* — animation is a “flip book”

pizza

just sits there for hedgehog to grab

phone

For now does nothing. Students can add code to it

Hedgehog: state, moving and eating pizza

- speed, lives and points track the state of the game
- act method:

```
public void act() {  
    moveWithKeys();  
    showStatus();  
    hitPizza();  
    hitBalloon();  
    testEndGame();  
    testWinGame();  
}
```

moveWithKeys: 0, 0 is TOP LEFT

```
public void moveWithKeys(){
    if (Greenfoot.isKeyDown("right")) {
        setLocation(getX()+speed, getY());
    }
    if (Greenfoot.isKeyDown("Left")) {
        setLocation(getX()-speed, getY());
    }
    if (Greenfoot.isKeyDown("Up")) {
        setLocation(getX(), getY()-speed);
    }
    if (Greenfoot.isKeyDown("Down")) {
        setLocation(getX(), getY()+speed);
    }
}
```

hitting pizza and balloons

```
public void hitPizza(){  
    if (isTouching (pizza.class) ){  
        removeTouching (pizza.class);  
        addPoints();  
    }  
}
```

```
public void hitBalloon(){  
    if (isTouching (balloon.class) ){  
        setLocation(300,100);  
        loseLife();  
    }  
}
```

Only remove the
pizza we are
touching, not all of
them

setLocation() warps
the Hedgehog by
immediately moving
it

showStatus

```
public void showStatus(){  
    getWorld().showText("Lives"+Lives, 50, 30);  
    getWorld().showText("Points"+Points, 50, 10);  
}
```

Let's do a demo

